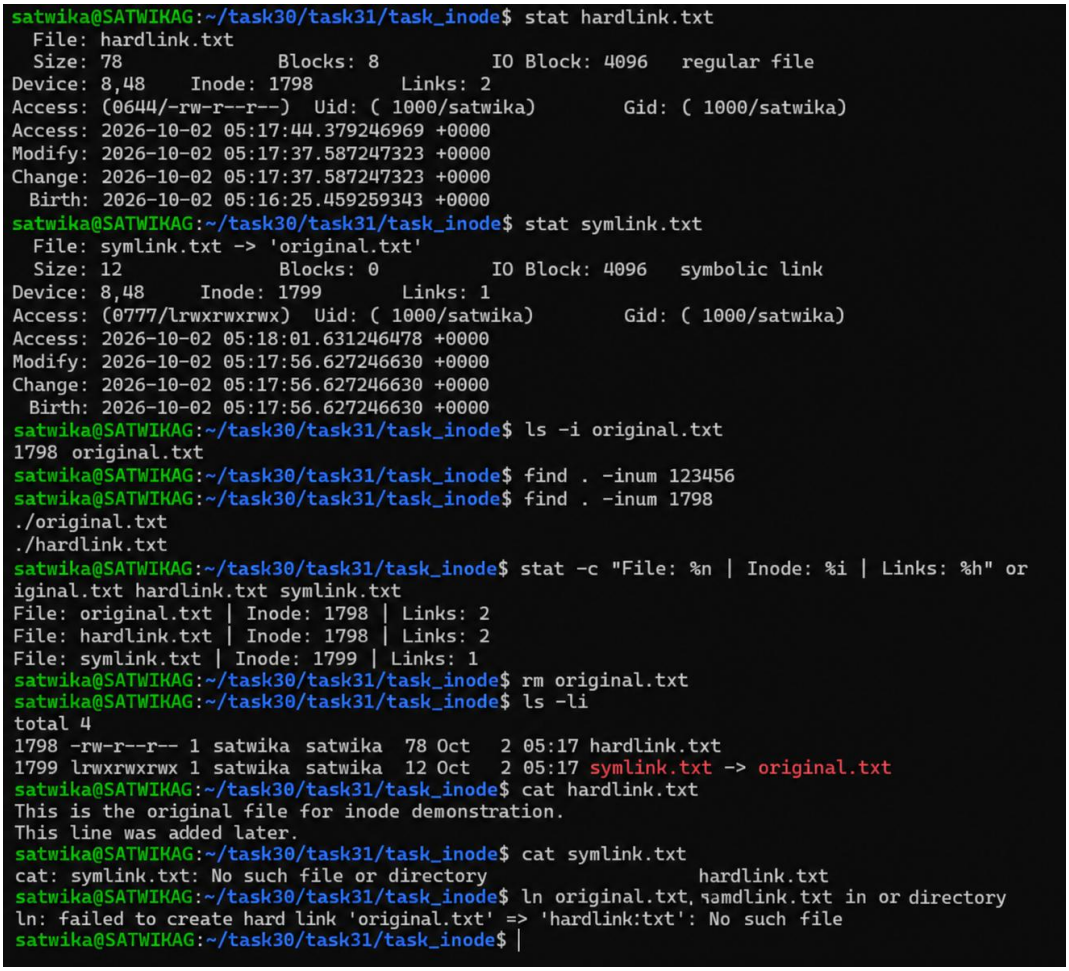
Practical Week – 10

Task 1 - Investigate inode structures using commands such as `ls -i`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

```
satwika@SATWIKAG:~/task30/task31$ mkdir task_inode
satwika@SATWIKAG:~/task30/task31$ cd task_inode
satwika@SATWIKAG:~/task30/task31/task_inode$ pwd
/home/satwika/task30/task31/task_inode
satwika@SATWIKAG:~/task30/task31/task_inode$ echo "This is the original file for inode dem
onstration." > original.txt
satwika@SATWIKAG:~/task30/task31/task_inode$ ls -i original.txt
1798  original.txt
satwika@SATWIKAG:~/task30/task31/task_inode$ stat original.txt
  File: original.txt
  Size: 51              Blocks: 8          IO Block: 4096   regular file
Device: 8,48    Inode: 1798        Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/satwika)      Gid: ( 1000/satwika)
Access: 2026-10-02 05:16:25.459259343 +0000
Modify: 2026-10-02 05:16:25.459259343 +0000
Change: 2026-10-02 05:16:25.459259343 +0000
 Birth: 2026-10-02 05:16:25.459259343 +0000
satwika@SATWIKAG:~/task30/task31/task_inode$ ln original.txt hardlink.txt
satwika@SATWIKAG:~/task30/task31/task_inode$ ls -li original.txt hardlink.txt
1798 -rw-r--r-- 2 satwika satwika 51 Oct  2 05:16 hardlink.txt
1798 -rw-r--r-- 2 satwika satwika 51 Oct  2 05:16 original.txt
satwika@SATWIKAG:~/task30/task31/task_inode$ cat hardlink.txt
This is the original file for inode demonstration.
satwika@SATWIKAG:~/task30/task31/task_inode$ echo "This line was added later." >> original
.txt
satwika@SATWIKAG:~/task30/task31/task_inode$ cat original.txt
This is the original file for inode demonstration.
This line was added later.
satwika@SATWIKAG:~/task30/task31/task_inode$ cat hardlink.txt
This is the original file for inode demonstration.
This line was added later.
satwika@SATWIKAG:~/task30/task31/task_inode$ ln -s original.txt symlink.txt
satwika@SATWIKAG:~/task30/task31/task_inode$ ls -li original.txt hardlink.txt symlink.txt
1798 -rw-r--r-- 2 satwika satwika 78 Oct  2 05:17 hardlink.txt
1798 -rw-r--r-- 2 satwika satwika 78 Oct  2 05:17 original.txt
1799 lrwxrwxrwx 1 satwika satwika 12 Oct  2 05:17 symlink.txt -> original.txt
satwika@SATWIKAG:~/task30/task31/task_inode$ cat symlink.txt
This is the original file for inode demonstration.
This line was added later.
satwika@SATWIKAG:~/task30/task31/task_inode$ stat original.txt
  File: original.txt
  Size: 78              Blocks: 8          IO Block: 4096   regular file
Device: 8,48    Inode: 1798        Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/satwika)      Gid: ( 1000/satwika)
Access: 2026-10-02 05:17:44.379246969 +0000
Modify: 2026-10-02 05:17:37.587247323 +0000
Change: 2026-10-02 05:17:37.587247323 +0000
 Birth: 2026-10-02 05:16:25.459259343 +0000
```



Task 2 - Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and

implementation complexity.

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>

int main() {
    int fd;
    struct stat file_stat;
    char *mapped;

    /* Open the file */
    fd = open("input.txt", O_RDWR);

    if (fd == -1) {
        perror("open");
        return 1;
    }

    /* Get file size */
    if (fstat(fd, &file_stat) == -1) {
        perror("fstat");
        close(fd);
        return 1;
    }

    printf("File size: %ld bytes\n", file_stat.st_size);

    /* Map file into memory */
    mapped = mmap(NULL,
                  file_stat.st_size,
                  PROT_READ | PROT_WRITE,
                  MAP_SHARED,
                  fd,
                  0);

    if (mapped == MAP_FAILED) {
        perror("mmap");
        close(fd);
        return 1;
    }

    /* Read file using memory mapping */
    printf("\nFile contents using mmap():\n");
    write(STDOUT_FILENO, mapped, file_stat.st_size);

    /* Modify file through memory mapping */
    const char *text = "Modified using mmap()";

    if (file_stat.st_size >= strlen(text)) {
        memcpy(mapped, text, strlen(text));
    }

    /* Make changes visible */
    msync(mapped, file_stat.st_size, MS_SYNC);

    /* Unmap memory */
    munmap(mapped, file_stat.st_size);

    close(fd);

    printf("\n\nFile successfully modified using mmap().\n");

    return 0;
}
```

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

int main() {
    int fd;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;

    fd = open("input.txt", O_RDONLY);

    if (fd == -1) {
        perror("open");
        return 1;
    }

    while ((bytes_read = read(fd, buffer, BUFFER_SIZE)) > 0) {
        write(STDOUT_FILENO, buffer, bytes_read);
    }

    close(fd);

    return 0;
}
```

```
satwika@SATWIKAG:~/task30/task31/task_inode$  mkdir task_mmap
cd task_mmap
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap$ echo "Hello! This is a test file fo
r mmap." > input.txt
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap$ cat input.txt
Hello! This is a test file for mmap.
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap$ nano mmap_file.c


satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap$ gcc mmap_file.c -o mmap_file
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap$ ls
input.txt  mmap_file  mmap_file.c
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap$ ./mmap_file
File size: 37 bytes

File contents using mmap():
Hello! This is a test file for mmap.

File successfully modified using mmap().
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap$ cat input.txt
Modified using mmap() file for mmap.
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap$ mmap()
> msync()
-bash: syntax error near unexpected token 'msync'
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap$ nano traditional_io.c
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap$ gcc traditional_io.c -o traditional
_io
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap$ ./traditional_io
Modified using mmap() file for mmap.
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap$
```

Task 2 - Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

```c
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>

int main() {

    int fd;
    struct stat file_info;
    char *mapped;

    /* Open file for reading and writing */
    fd = open("input.txt", O_RDWR);

    if (fd == -1) {
        perror("Error opening file");
        return 1;
    }

    /* Get file size */
    if (fstat(fd, &file_info) == -1) {
        perror("Error getting file information");
        close(fd);
        return 1;
    }

    printf("File size: %ld bytes\n", file_info.st_size);

    /* Map file into memory */
    mapped = mmap(NULL,
                  file_info.st_size,
                  PROT_READ | PROT_WRITE,
                  MAP_SHARED,
                  fd,
                  0);

    if (mapped == MAP_FAILED) {
        perror("mmap failed");
        close(fd);
        return 1;
    }

    /* Read file using mmap */
    printf("\nFile contents using mmap():\n");
    write(STDOUT_FILENO, mapped, file_info.st_size);

    /* Modify file through mapped memory */
    const char *new_text = "File modified using mmap().";

    if (strlen(new_text) <= file_info.st_size) {
        memcpy(mapped, new_text, strlen(new_text));
    }

    /* Synchronize changes with file */
    msync(mapped, file_info.st_size, MS_SYNC);

    /* Remove mapping */
    munmap(mapped, file_info.st_size);

    close(fd);

    printf("\n\nFile successfully modified using mmap().\n");

    return 0;
}
```

```c
#include <fcntl.h>
#include <unistd.h>
#define BUFFER_SIZE 1024
int main() {
    int fd;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;
  fd = open("largefile.txt", O_RDONLY);
    if (fd == -1) {
        perror("Error opening file");
        return 1;
    }
printf("File contents using read():\n");
    while ((bytes_read = read(fd, buffer, BUFFER_SIZE)) > 0) {
        write(STDOUT_FILENO, buffer, bytes_read);
    }
    if (bytes_read == -1) {
        perror("Error reading file");
    }
    close(fd);
    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

int main() {

    int fd;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;

    fd = open("input.txt", O_RDONLY);

    if (fd == -1) {
        perror("Error opening file");
        return 1;
    }

    printf("File contents using read():\n");

    while ((bytes_read = read(fd, buffer, BUFFER_SIZE)) > 0) {
        write(STDOUT_FILENO, buffer, bytes_read);
    }

    if (bytes_read == -1) {
        perror("Error reading file");
    }

    close(fd);

    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

int main() {

    int fd;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;
fd = open("largefile.txt", O_RDWR);
 if (fd == -1) {
        perror("Error opening file");
        return 1;
    }

    printf("File contents using read():\n");

    while ((bytes_read = read(fd, buffer, BUFFER_SIZE)) > 0) {
        write(STDOUT_FILENO, buffer, bytes_read);
    }

    if (bytes_read == -1) {
        perror("Error reading file");
    }

    close(fd);

    return 0;
}
```

```
satwika@SAvA@AMMG:~/task30/task31/task_inode/task_mmap$ mkdir task2_mmap
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap$ cd task2_mmap
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap/task2_mmap$ echo "Hello! This is a f
ile for mmap demonstration." > input.txt
satwika@SAvA@AMMG:~/task30/task31/task_inode/task_mmap/task2_mmap$ cat input.txt
Hello! This is a file for mmap demonstration.
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap/task2_mmap$ nano mmap_program.c

satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap/task2_mmap$ gcc mmap_program.c -o mm
ap_program
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap/task2_mmap$ ls
input.txt  mmap_program  mmap_program.c
satwika@SAvA@AMMG:~/task30/task31/task_inode/task_mmap/task2_mmap$ ./mmap_program
File size: 46 bytes

File contents using mmap():
Hello! This is a file for mmap demonstration.

File successfully modified using mmap().
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap/task2_mmap$ cat input.txt
File modified using mmap() map demonstration.
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap/task2_mmap$ nano traditional_program
.c
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap/task2_mmap$ gcc traditional_program.
c -o traditional_program
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap/task2_mmap$ ./traditional_program
File contents using read():
File modified using mmap().map demonstration.
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap/task2_mmap$ dd if=/dev/zero of=large
file.txt bs=1M count=100
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.0888376 s, 1.2 GB/s
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap/task2_mmap$ cp traditional_program.c
traditional_large.c
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap/task2_mmap$ nano traditional_large.c
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap/task2_mmap$ gcc traditional_large.c
-o traditional_large
traditional_large.c:3:10: fatal error: stdlib.h0: No such file or directory
    3 | #include <stdlib.h0>
      |          ^~~~~~~~~~~
compilation terminated.
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap/task2_mmap$ cp mmap_program.c mmap_l
arge.c
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap/task2_mmap$ nano mmap_large.c
```

```
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap/task2_mmap$ gcc mmap_large.c -o mmap
_large
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap/task2_mmap$ time ./mmap_large > mmap
_output.txt

real    0m0.717s
user    0m0.032s
sys     0m0.684s
satwika@SATWIKAG:~/task30/task31/task_inode/task_mmap/task2_mmap$
```